Lab 1: Version control with Git

Build, fix and share a small app

Mohammed Said Belaid

Duration: 1h30, in pairs

Part 0: Setup (5 min)

```
git config --global user.name "Firstname Lastname"
git config --global user.email "you@example.com" # your GitHub e-mail
git config --global init.defaultBranch main
git config --global alias.lg "log --oneline --graph --all --decorate"
ssh -T git@github.com # fails? ssh-keygen -t ed25519, then paste
                      # ~/.ssh/id_ed25519.pub in GitHub > Settings > SSH keys
```

Part 1: Start the project (15 min)

```
mkdir -p ~/devops/shop && cd ~/devops/shop
git init
```

Create `cart.py`:

```
TAX_RATE = 0.19

def average(prices):
    return sum(prices) / len(prices)

def total(prices):
    """Cart total, taxes included."""
    return round(sum(prices) * (1 + TAX_RATE), 2)
```

and `main.py`:

```
from cart import total, average

cart = [10, 20]
print("Total:", total(cart))
print("Average:", average(cart))
```

```
python3 main.py # Total: 35.7
git status
git add cart.py main.py
git commit -m "Add cart total and average"
git branch -m main # only if git status said "On branch master" (Git < 2.28)
```

Running the app created `__pycache__/`, and you need a local `.env` for secrets. Create `.env` containing the line `DB_PASSWORD=secret`, then:

```
git status # __pycache__/ and .env appear
```

Create `.gitignore`:

```
__pycache__/
.env
.venv/
```

```
git status # only .gitignore is left
git add .gitignore
git commit -m "Ignore Python cache and local secrets"
```

Part 2: A feature, and an urgent bug (20 min)

Start a feature on its own branch:

```
git switch -c feature/discount
```

Append to `cart.py`:

```
def apply_discount(prices, percent):
    return [p * (1 - percent / 100) for p in prices]
```

```
git diff # review before committing
git commit -am "Add percentage discount"
```

A user reports a crash: an empty cart breaks the app.

```
git switch main # your feature is safe on its branch
python3 -c "from cart import average; average([])" # ZeroDivisionError
git switch -c fix/empty-cart
```

Fix `average` in `cart.py`:

```
def average(prices):
    if not prices:
        return 0
    return sum(prices) / len(prices)
```

```
git commit -am "Fix crash on empty cart"
git switch main
git merge fix/empty-cart # 1st merge
git merge feature/discount # 2nd merge (accept the default message)
git lg
python3 main.py
```

Part 3: A real conflict (15 min)

Two developers change the tax rate at the same time.

```
git switch -c update-tax
```

In `cart.py`, set `TAX_RATE = 0.09`, then:

```
git commit -am "Use reduced tax rate"
git switch main
```

In `cart.py`, set `TAX_RATE = 0.21`, then:

```
git commit -am "Update tax rate for 2027"
git merge update-tax # CONFLICT
git status
```

Open `cart.py`, keep the right value, delete the three conflict-marker lines, then:

```
python3 main.py # always check the code still runs
git add cart.py
git commit # concludes the merge
```

Now update a branch with **rebase** instead of merge:

```
git switch -c feature/receipt
```

Add the line `print("Thank you!")` at the end of `main.py`, then:

```
git commit -am "Print thank-you line"
git switch main
```

Create `README.md` containing `# Shop`, then:

```
git add README.md && git commit -m "Add README"
git switch feature/receipt
git lg # note the ID of "Print thank-you line"
git rebase main
git lg
```

Part 4: Undo mistakes (10 min)

```
git switch main && git merge feature/receipt
```

a) Broke a file, not committed: get it back. Replace the whole content of `cart.py` with `oops` and save, then:

```
git restore cart.py
```

b) Forgot a file in the last commit (not pushed). Create `test_cart.py` containing `# Tests coming in lab 2`, then:

```
git add test_cart.py && git commit --amend --no-edit
```

c) A bad commit is already shared: undo it with a new commit. In `average` (`cart.py`), change `return 0` into `return None`, then:

```
git commit -am "Return None for empty cart"
python3 main.py # still fine... but average([]) now breaks callers
git revert HEAD --no-edit
git lg
```

Part 5: Share it on GitHub (25 min, together)

This repository is your pair's project for the whole semester.

A: create an empty **private** repository `devops-<nameA>-<nameB>` on GitHub (no README), invite **B** and the teacher, then:

```
cd ~/devops/shop
git remote add origin git@github.com:<A-user>/devops-<nameA>-<nameB>.git
git push -u origin main
```

B: clone it:

```
cd ~/devops && git clone git@github.com:<A-user>/devops-<nameA>-<nameB>.git
```

5.1 Both on main: feel the problem

Both students change the first line of `README.md` differently, commit, then push. **A** pushes first.

```
# B: push is rejected
git pull # conflict: resolve it, then
git add README.md && git commit && git push
# A
git pull && git lg
```

5.2 One branch per task

```
# A: add a function count(prices) to cart.py
git switch -c feature/count
git commit -am "Add item count" && git push -u origin feature/count

# B: write TEAM.md with both names
git switch -c docs/team
git add TEAM.md && git commit -m "Add team file" && git push -u origin docs/team
```

One student merges both branches into `main` and pushes; the other pulls. Then tag:

```
git switch main && git pull
git merge origin/feature/count && git merge origin/docs/team
git push
git tag -a lab1 -m "End of lab 1" && git push origin lab1
```

Cheat sheet

Command	Purpose
`git status` / `git diff [--staged]`	What changed / what is staged
`git add <file>` / `git commit -m`	Stage / record a change
`git switch [-c] <branch>`	Change (create) branch
`git merge <branch>`	Merge a branch into the current one
`git rebase <branch>`	Replay my commits on top of a branch
`git restore [--staged] <file>`	Discard a change / unstage
`git commit --amend`	Fix the last commit (not pushed!)
`git revert <id>`	Undo a pushed commit
`git pull` / `git push`	Get / send commits
`git lg`	History as a graph